

Learning File 3: The Code Implementation

This file explains how the logic is written in Python and TypeScript. Open the files as you read this.

1. Backend: rag_service.py (The Core)

Class: RiskAssessment (Lines 33-35)

- This uses "Pydantic". It defines the exact shape of data we want from the AI.
- If the AI returns a string instead of a number for "risk_score", this code will throw an error. It guarantees type safety.

Function: ingest_document (Lines 39-72)

- Input: A raw file.
- Output: A string (doc_id).
- Key Line: `vector_store.add_documents(documents=splits)`
 - This single line does the heavy lifting: it calculates the vectors for all chunks and saves them to the disk in the `./chroma_db` folder.

Function: analyze_document (Lines 74-141)

- Input: doc_id, checklist.
- Output: A dictionary (The Report).
- Key Line: `retriever.invoke(item)`
 - This performs the Vector Search (Cosine Similarity).
- Key Line: `chain.invoke(...)`
 - This sends the request to Groq. "Chain" is a LangChain concept. It links the Prompt -> LLM -> Output Parser together.

2. Backend: main.py (The API)

- `app.add_middleware(...)`: This handles CORS. It allows your Frontend (running on port 5173) to talk to your Backend (running on port 8001). Without this, the browser would block the request for security.
- `async def`: Notice the functions are "async". This means the server can handle other requests while waiting for the file to upload. It doesn't freeze.

3. Frontend: App.tsx (The State Machine)

- `useState`: React's memory.
 - `docId`: Remembers the ID of the file you uploaded.
 - `report`: Remembers the result from the audit.
- `useMutation` (from React Query):
 - This is a smart way to handle API calls.
 - It gives us `isPending` (True when loading), `isError` (True when failed), and `isSuccess`.
 - We use this to show the "Auditing..." spinner button.

4. Frontend: Components

- Docket.tsx:
 - Uses a hidden `<input type="file">`. When you click the big box, it actually clicks this hidden input.
- AuditorPanel.tsx:
 - The Gauge Chart is just an SVG circle.
 - We use `strokeDashoffset` to animate the circle filling up based on the `riskScore`.
 - Logic: `offset = circumference - (circumference * score / 100)`.

5. Summary of Execution

6. You run `docker-compose up`.
7. Docker builds the Python container and the Node.js container.
8. Nginx (in frontend container) serves the React app.
9. Uvicorn (in backend container) starts the API.
10. They wait for your input.